

Assignment 1\A1_Code_3.py

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib.animation import FuncAnimation
4
5 # Initializing Variables
6 x_start = 0
7 x_end = 1
8
9 t_start = 0
10 t_end = 1
11
12 nx = 100 #number of grid points
13 CFL = 0.8
14
15 dx = (x_end-x_start)/nx
16 x = np.arange(x_start,x_end,dx) #establishing discretized x locations
17
18 u0 = np.exp(-100*(x-0.5)**2) #Calculating initial condition on the x grid
19
20 u_n = np.zeros(len(x)) #Initializing current time grid point velocity vector
21 u_np1 = np.zeros(len(x)) #Initializing next time grid point velocity vector
22
23
24 u_n = u0.copy() #Assigning the first current time grid point velocity vector to the initial
    condition
25
26 dt = (CFL*dx)/max(np.abs(u0)) #Calculating the most restrictive dt I can use based on a
    constant CFL number and the maximum velocity in my domain
27
28 u_history = [] #Initializing a time history velocity vector to store information for
    animations
29 t_history = []
30
31 t_current = 0
32
33 while t_current < t_end: #Loop through time until we have simulated up until the end time
34     for i in range(nx): #Loop through the grid points
35         if i == 0:
36             u_np1[i] = u_n[i] - (u_n[i] * dt/dx)*(u_n[i] - u_n[-1])
37         else:
38             u_np1[i] = u_n[i] - (u_n[i] * dt/dx)*(u_n[i] - u_n[i-1])
39
40     t_current = t_current + dt
41     u_n = u_np1.copy()
42     dt = (CFL*dx)/max(np.abs(u_n))
43
44     u_history.append(u_n.copy())
45     t_history.append(t_current)
```

```
46
47 u_history = np.array(u_history)
48
49 u_history_nc = u_history.copy()
50
51 Q_nc = np.sum(u_history_nc*dx,axis=1)
52 t_history_nc = t_history
53
54 # plt.plot(x,u_np1)
55 # plt.xlabel("x")
56 # plt.ylabel("u")
57 # plt.title("t = 1")
58 # plt.show()
59
60
61 # plt.plot(x,u_history[0,:],"k",label="Initial Solution")
62 # plt.plot(x,u_history[-1,:],"r",label="Final Solution")
63 # plt.xlabel("x")
64 # plt.ylabel("u")
65 # plt.legend()
66 # plt.title("Initial and Final Solutions")
67
68 # plt.show()
69
70
71 # fig, ax = plt.subplots()
72
73 # line, = ax.plot(x, u_history[0, :])
74 # ax.set_xlim(x_start, x_end)
75 # ax.set_ylim(np.min(u_history), np.max(u_history))
76
77 # def update(frame):
78 #     line.set_ydata(u_history[frame, :])
79 #     ax.set_title(f"Time = {t_history[frame]:.3f} s")
80 #     return line,
81
82 # animation = FuncAnimation(
83 #     fig,
84 #     update,
85 #     frames=len(u_history),
86 #     interval=50
87 # )
88
89 # plt.show()
90
91
92 # Initializing Variables
93 x_start = 0
94 x_end = 1
95
```

```

96  t_start = 0
97  t_end = 1
98
99  nx = 100 #number of grid points
100 CFL = 0.8
101
102 dx = (x_end-x_start)/nx
103 x = np.arange(x_start,x_end,dx) #establishing discretized x locations
104
105 u0 = np.exp(-100*(x-0.5)**2) #Calculating initial condition on the x grid
106
107 u_n = np.zeros(len(x)) #Initializing current time grid point velocity vector
108 u_np1 = np.zeros(len(x)) #Initializing next time grid point velocity vector
109
110
111 u_n = u0.copy() #Assigning the first current time grid point velocity vector to the initial
    condition
112
113 dt = (CFL*dx)/max(np.abs(u0)) #Calculating the most restrictive dt I can use based on a
    constant CFL number and the maximum velocity in my domain
114
115 u_history = [] #Initializing a time history velocity vector to store information for
    animations
116 t_history = []
117
118 t_current = 0
119
120 while t_current < t_end: #Loop through time until we have simulated up until the end time
121     for i in range(nx): #Loop through the grid points
122         if i == 0:
123             u_np1[i] = u_n[i] - (dt/(2*dx))*(u_n[i]**2 - u_n[-1]**2)
124         else:
125             u_np1[i] = u_n[i] - (dt/(2*dx))*(u_n[i]**2 - u_n[i-1]**2)
126
127         t_current = t_current + dt
128         u_n = u_np1.copy()
129         dt = (CFL*dx)/max(np.abs(u_n))
130
131         u_history.append(u_n.copy())
132         t_history.append(t_current)
133
134 u_history = np.array(u_history)
135
136 u_history_c = u_history.copy()
137
138 Q_c = np.sum(u_history_c*dx,axis=1)
139 t_history_c = t_history
140
141
142 Q0 = np.sum(u0*dx)
143 # plt.plot(x,u_np1)

```

```
144 # plt.xlabel("x")
145 # plt.ylabel("u")
146 # plt.title("t = 1")
147 # plt.show()
148
149
150 # plt.plot(x,u_history_nc[-1,:],"k",label="Non Conservative Solution")
151 # plt.plot(x,u_history_c[-1,:],"r",label="Conservative Solution")
152 # plt.xlabel("x")
153 # plt.ylabel("u")
154 # plt.legend()
155 # plt.title("Non Conservative vs Conservative Solutions")
156
157 # plt.show()
158
159
160 # fig, ax = plt.subplots()
161
162 # line, = ax.plot(x, u_history[0, :])
163 # ax.set_xlim(x_start, x_end)
164 # ax.set_ylim(np.min(u_history), np.max(u_history))
165
166 # def update(frame):
167 #     line.set_ydata(u_history[frame, :])
168 #     ax.set_title(f"Time = {t_history[frame]:.3f} s")
169 #     return line,
170
171 # animation = FuncAnimation(
172 #     fig,
173 #     update,
174 #     frames=len(u_history),
175 #     interval=50
176 # )
177
178 # plt.show()
179
180 plt.plot(t_history_nc,Q_nc,"k",label="Non Conservative")
181 plt.plot(t_history_c,Q_c,"r",label="Conservative ")
182 plt.plot(0,Q0,"bo",label="Q0")
183
184 plt.xlabel("t")
185 plt.ylabel("Q")
186 plt.legend()
187 plt.title("Non Conservative vs Conservative")
188
189 plt.show()
```